

0. Prerequisites

bash

Local tools you need installed

terraform -version # >= 1.5

kubectl version --client

helm version

dvc --version

docker --version

aws --version

You also need:

- An AWS account with permission to create VPC/EC2/IAM resources
- An existing EC2 key pair: `aws ec2 create-key-pair --key-name mlops-key --query 'KeyMaterial' --output text > mlops-key.pem && chmod 400 mlops-key.pem`
- An S3 bucket already created: `aws s3 mb s3://your-mlops-bucket --region ap-south-1`
- A container registry you can push to (Docker Hub, ECR, GHCR)

Unzip the project and cd into it:

bash

unzip mlops-platform.zip -d mlops-platform && cd mlops-platform

1. Provision the cluster (Terraform)

bash

cd infra/terraform

cp terraform.tfvars.example terraform.tfvars

Edit terraform.tfvars — set at minimum:

hcl

key_name = "mlops-key"

allowed_ssh_cidr = "<your-ip>/32"

kubeconfig_bucket = "your-mlops-bucket"

dvc_bucket = "your-mlops-bucket" # can be the same bucket

bash

terraform init

terraform apply # type "yes" when prompted

This takes a few minutes to create the VPC/IAM/instances, then the instances bootstrap themselves in the background.

2. Wait for bootstrap, then pull the kubeconfig

```
bash
```

```
ssh -i ../../mlops-key.pem ubuntu@$(terraform output -raw master_public_ip) \  
  'tail -f /var/log/mlops-bootstrap.log'
```

```
# Ctrl-C once you see "Master bootstrap complete."
```

```
bash
```

```
terraform output -raw fetch_kubeconfig | bash
```

```
export KUBECONFIG=$(pwd)/kubeconfig.yaml
```

```
kubectl get nodes
```

Expect 1 master + N workers, all Ready within ~60 seconds of joining. If a worker shows NotReady, give Calico another minute to finish setting up pod networking.

3. Install the platform components

Back to the repo root, keep KUBECONFIG exported:

```
bash
```

```
cd ../../ # back to mlops-platform/
```

```
bash platform/01-install-cert-manager.sh
```

```
bash platform/02-install-argo-workflows.sh
```

```
bash platform/03-install-airflow.sh
```

```
bash platform/04-install-kserve.sh
```

Run them in this order — Argo and Airflow need their namespaces first, and KServe needs cert-manager already healthy. Each script waits for its own rollout before exiting, so they're safe to run sequentially without manual polling.

Check everything came up:

```
bash
```

```
kubectl get pods -n cert-manager
```

```
kubectl get pods -n argo
```

```
kubectl get pods -n airflow
```

```
kubectl get pods -n kserve
```

4. Set up DVC and build the training image

```
bash
```

```

bash dvc/setup.sh your-mlops-bucket dvc-store
dvc repro      # confirm the pipeline runs locally first
dvc push
bash
docker build -t your-registry/mlops-training:latest -f pipelines/Dockerfile .
docker push your-registry/mlops-training:latest

```

5. Fill in the placeholders

Three files reference your bucket/image/region — edit each:

File	What to set
workflows/training-workflow-template.yaml	image and dvc-bucket default values
airflow_dags/mlops_training_pipeline.py	TRAINING_IMAGE, DVC_BUCKET constants near the top
kserve/inferenceservice.yaml	storageUri bucket name, AWS_DEFAULT_REGION (only matters if you apply this manually)

6. Register the Argo WorkflowTemplate

```

bash
kubectl apply -f workflows/training-workflow-template.yaml
kubectl -n mlops get workflowtemplate mlops-training

```

7. Get the DAG into Airflow

platform/airflow/values.yaml is configured for git-sync. Either:

Option A — push this repo to your own git remote and update dags.gitSync.repo in platform/airflow/values.yaml, then re-run:

```

bash
helm upgrade airflow apache-airflow/airflow -n airflow -f platform/airflow/values.yaml --version 1.13.1

```

Option B — quick local test, no git needed:

```

bash
kubectl -n airflow get pods -l component=scheduler
kubectl -n airflow cp airflow_dags/mlops_training_pipeline.py \
  <scheduler-pod-name>:/opt/airflow/dags/mlops_training_pipeline.py

```

8. Trigger it

```

bash

```

```
kubectl -n airflow get svc airflow-webserver -o jsonpath='{.spec.ports[0].nodePort}'
```

Open <http://<any-node-public-ip>:<that-nodeport>>, unpause `mlops_training_pipeline`, hit the trigger (▶) button.

9. Watch it run

```
bash
```

```
kubectl -n mlops get workflows -w
```

Once it shows Succeeded, check the deploy:

```
bash
```

```
kubectl -n mlops get inferencesservice mlops-model
```

```
kubectl -n mlops get pods -l serving.kserve.io/inferencesservice=mlops-model
```

10. Test the served model

```
bash
```

```
kubectl -n mlops port-forward svc/mlops-model-predictor 8080:80
```

In another terminal:

```
bash
```

```
curl -X POST localhost:8080/v1/models/mlops-model:predict \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"instances": [[14.1, 20.0, 90.0, 600.0, 0.1, 0.1, 0.05, 0.03, 0.18, 0.06, 0.4, 1.2, 2.5, 40.0, 0.006, 0.02, 0.02, 0.01, 0.02, 0.003, 16.0, 25.0, 105.0, 800.0, 0.14, 0.25, 0.27, 0.11, 0.29, 0.08]]}'
```

11. Tear down (when you're done with the PoC)

```
bash
```

```
cd infra/terraform
```

```
terraform destroy
```